TicketStar

Propuesta de arquitectura · Entrega 1

Programación Distribuida y Componentes

Francisco Aguiar · Rodrigo Rodriguez

El problema

- Cuando sale a la venta un evento grande, entra mucha más gente de la que hay entradas.
- Si el sistema no está preparado se cae, o vende la misma butaca dos veces.
- En la puerta hay que controlar que una entrada no se use dos veces.
- Un teatro o un club tiene su página y sus clientes: le falta la parte de la venta.

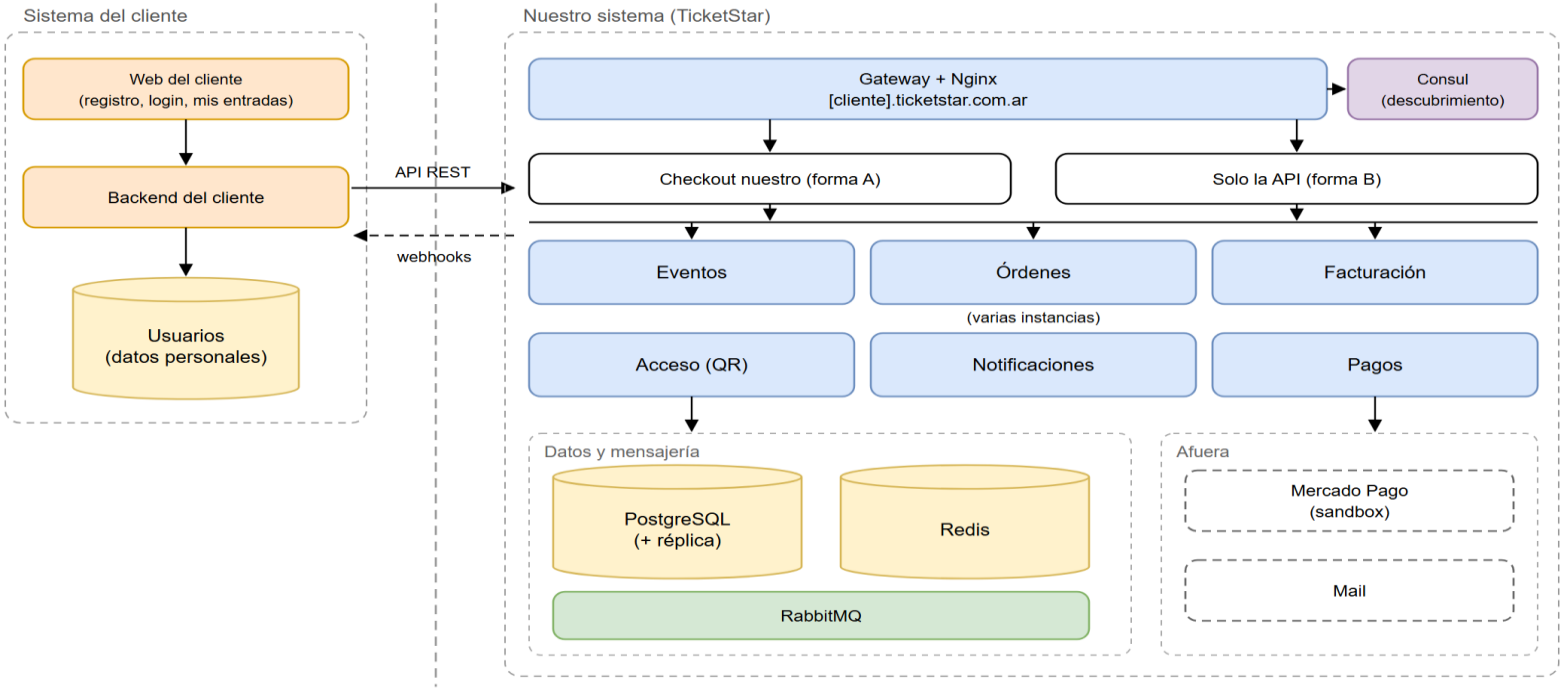
Ticketmaster, 2022

El sitio colapsó con la preventa de Taylor Swift.

Nuestra idea

Ofrecer la venta de entradas como servicio, conectado al sistema de cada cliente.

Venta de entradas como servicio



El cliente nos llama por la API y nosotros le avisamos por webhooks cuando pasa algo.

Dos formas de conectarse

Forma A: checkout nuestro

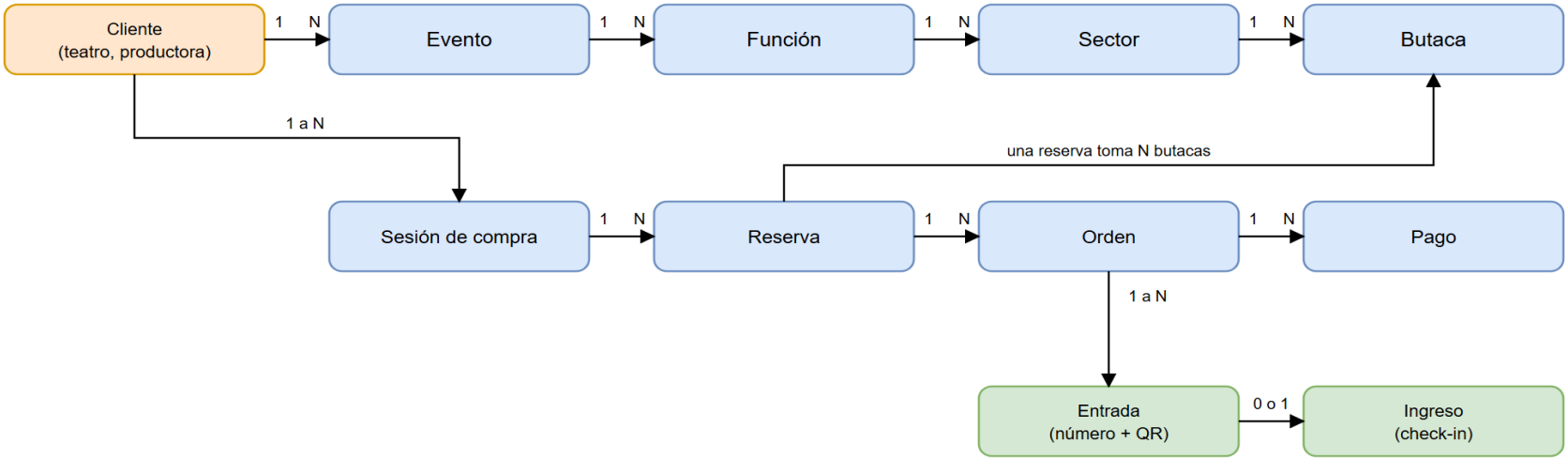
El cliente manda a la persona a una página nuestra, con su logo, y la recibe de vuelta cuando terminó. Es la más fácil de integrar. La va a usar el Teatro Avenida.

Forma B: solo la API

El cliente hace su propia pantalla de compra y nos usa solo por la API. Tiene más control pero programa más. La va a usar el Club Atlético.

Las dos usan la misma API: lo único que cambia es quién dibuja la pantalla.

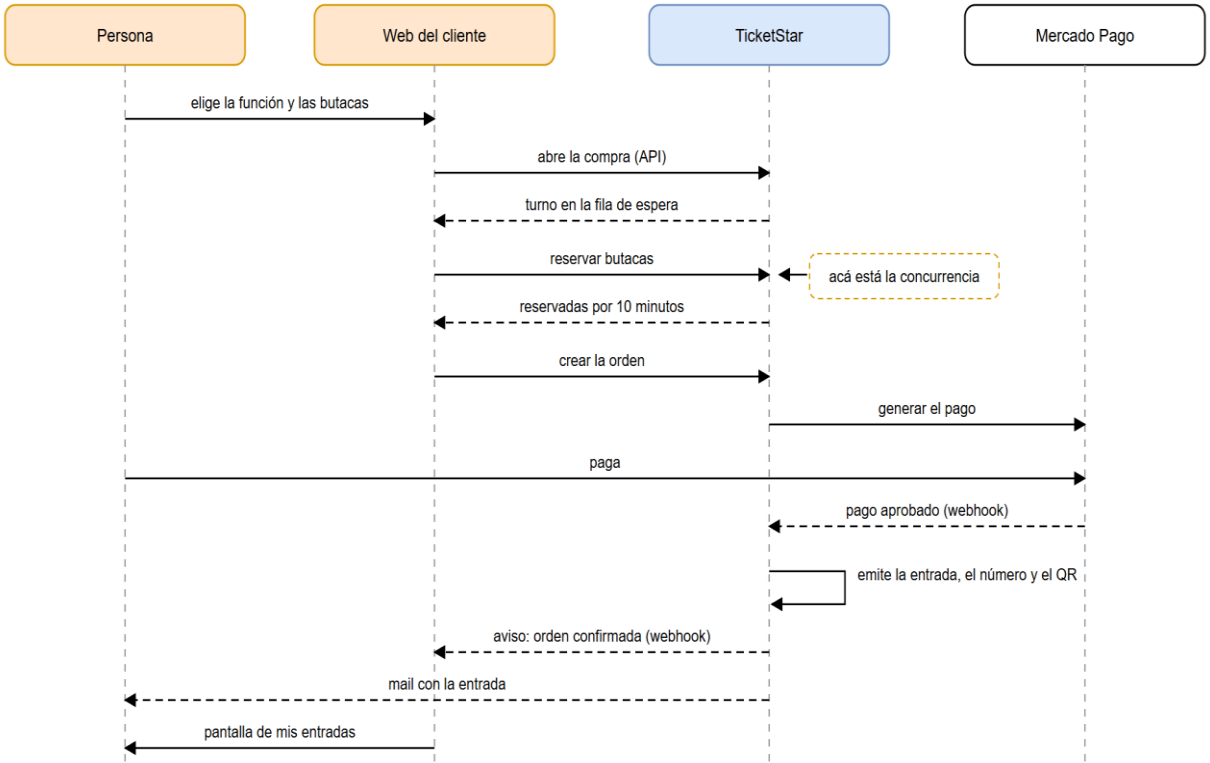
Vista lógica



Además, por cada cliente guardamos los avisos que le mandamos (webhooks) y el consumo del período para facturarle.

- Cada dato pertenece a un cliente y no se mezcla con el de otro.
- Los datos personales de los compradores quedan del lado del cliente.

Vista de procesos



Síncrono (REST)

Reservar la butaca: la persona espera la respuesta.

Asíncrono

Aviso de pago, mail con la entrada y aviso al cliente.

Concurrencia

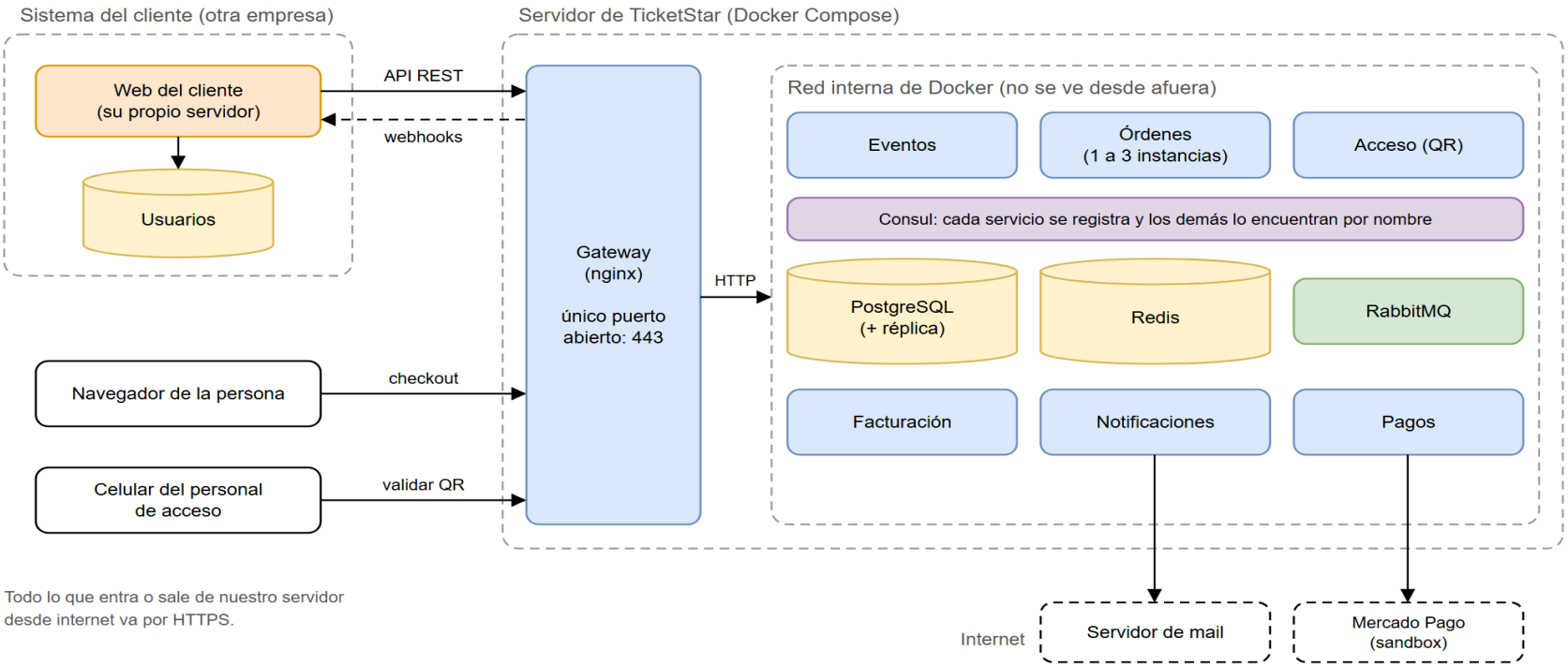
Dos personas, la misma butaca: una sola operación en la base.

Vista de desarrollo

```
ticketstar/
  docker-compose.yml
  gateway/
  servicios/
    eventos/   ordenes/
    pagos/     acceso/
    notificaciones/
    facturacion/
  contratos/          (OpenAPI)
  clientes-demo/
    teatro-avenida/
    club-atletico/
  pruebas/            (k6)
  docs/
```

- Un solo repositorio, con una carpeta por servicio.
- Primero el contrato de la API, después el código.
- Nuestros servicios en Python con FastAPI.
- Los clientes de prueba en Node, otro lenguaje a propósito.
- Una rama por tarea y un tag por entrega.

Vista física



Once tipos de contenedor con Docker Compose. Desde afuera solo se llega al gateway.

Escenario: se abre la venta

1. El cliente abre la compra por la API.
2. La persona espera su turno en la fila.
3. Pide la butaca: si otro se la llevó, elige de nuevo.
4. Paga con Mercado Pago, con las credenciales del cliente.
5. Se emite la entrada con su QR firmado.
6. Le avisamos al cliente y mandamos el mail.

Lógica

Butaca, reserva, orden y entrada.

Procesos

Reserva por REST; lo demás, asíncrono.

Desarrollo

Órdenes, pagos, notificaciones y la API.

Física

Gateway, órdenes, Postgres y RabbitMQ.

Próximos pasos

Lo que viene

- Entrega 2: el esqueleto con los contenedores separados.
- Entrega 3: la reserva de butacas y la prueba de concurrencia.
- Entrega 4: el contrato de la API y el descubrimiento de servicios.

Lo que queremos consultar

- Si los clientes de prueba cuentan como parte de la entrega.
- Si el sistema del cliente sirve como segundo nodo del módulo 2.
- Si la pasarela de pago puede ser simulada.